RAGE: Robust Agentic Security Gateway for Text-to-SQL — Defending Against Multi-Turn Crescendo Attacks

Authors: RAGE Research Team
Venue: Global South AI Safety Hackathon, June 2026
Repository: rage-multiturn — Python 3.12, MIT License
arXiv anchor: Russinovich et al., 2404.01833 [cs.CR]

Abstract

Text-to-SQL interfaces that connect language models to relational databases expose an attack surface where an adversary can gradually migrate a legitimate session toward destructive queries without triggering stateless filters. Russinovich et al. demonstrated that Crescendo achieves 98–100% success rates on aligned models using exclusively benign prompts distributed across N turns. We present RAGE (Robust Agentic Security Gateway for Text-to-SQL), a four-layer framework with a stateful semantic filter, EWMA decision engine with a consecutive-warn ratchet, and a hardened SQL gateway. We integrate the Training-Center, an interactive environment for simulation and hyperparameter calibration. We introduce the AUC-D (Area Under the Curve of Degradation) and TRI (Temporal Resistance Index) metrics. We validate the system with 108 automated tests (100% passing) and the SCENARIO_CRESCENDO scenario, where RAGE blocks turns T4 and T5 through independent action across layers 3, 4, and the gateway.

---

1. Introduction and Theoretical Framework

1.1 The Text-to-SQL Attack Surface

Organizations increasingly deploy conversational agents capable of translating natural language into SQL over operational data [2, 3]. Unlike web forms or REST APIs with rigid validation schemas, these systems accept inputs of arbitrary length whose semantic intent is difficult to bound with static signatures. A user who maintains a multi-turn conversation possesses expressive freedom that is fundamentally incompatible with the least-privilege principle in database security.

Prior defences evaluate each turn in isolation: they scan the current prompt for known patterns (DROP TABLE, IGNORE PREVIOUS INSTRUCTIONS) and reject it if any signature matches. This stateless model is a direct corollary of the single-turn alignment benchmarks that dominate current research. Russinovich et al. state explicitly: "all current benchmarks focus solely on single-turn jailbreaks" [1, §1], and demonstrate that Crescendo evades these measures with ease.

1.2 Theory of Change: From Stateless Filters to Session-Based Defences

The central conceptual contribution of RAGE is the Theory of Change toward session-aware defences. A persistent adversary does not seek to trigger a single alarm; they build conversational momentum that colonises the model's context turn by turn. Neutralising this adversary requires three capabilities absent from stateless filters:

1. Baseline anchoring: measure accumulated deviation from turn T0, not only from the immediately preceding turn.
2. Session risk memory: accumulate moderate signals that, individually, do not exceed per-turn thresholds.
3. Deterministic action containment: validate the resulting SQL query regardless of what the semantic layers permit.

Formally, let $S = \langle q_0, q_1, \ldots, q_N \rangle$ be a sequence of prompts and $e_i \in \mathbb{R}^{2048}$ the L2-normalised embedding of turn i. We define turn-to-turn drift $\delta_i = \max(0, 1 - e_i \cdot e_{i-1})$ and cumulative drift $\Delta_i = \max(0, 1 - e_i \cdot e_0)$. A stateless filter with threshold τ blocks turn i iff $\delta_i > \tau$. In a Crescendo attack with step size $\varepsilon \ll \tau$, all turns satisfy $\delta_i = \varepsilon < \tau$, but $\Delta_N = \sum_{i=1}^{N} \varepsilon \approx N\varepsilon$ may far exceed τ for sufficiently large N.

Proposition 1 (Stateless Filter Blindness). A filter that evaluates exclusively δ_i is blind to gradual escalation trajectories whose semantic vector monotonically converges away from e_0 without abrupt jumps between consecutive turns. The defence requires computing Δ_i and maintaining a session risk score R_t with exponential memory.

---

2. Related Work and Crescendo Threat Analysis

2.1 Prior Literature

Crescendo (Microsoft, 2024). Russinovich, Salem, and Eldan [1] introduce a black-box, multi-turn jailbreak that uses exclusively benign, human-readable prompts. Its mechanism is grounded in the foot-in-the-door psychological principle: agreeing to a small initial request systematically increases compliance with subsequent, larger demands. On LLaMA-2 70b, the full sequence A → B → C achieves 99.9% success, while B alone achieves 36.2% and C only 17.3% — confirming that it is the trajectory, not the individual turn, that constitutes the attack.

Alignment failures and universal attacks. Zou et al. [5] demonstrate transferable adversarial attacks on aligned models; Pérez and Ribeiro [6] document Ignore Previous Prompt techniques. These works operate predominantly in the single-turn regime.

OWASP LLM Top 10. The OWASP taxonomy [4] identifies directly relevant vectors: LLM01 (Prompt Injection), LLM06 (Excessive Agency — excessive agent control over tools), and LLM08 (undue trust in untrusted inputs, including intent summaries generated by the attacker). RAGE explicitly maps each layer to these identifiers.

Crescendomation. Russinovich et al. report that the automated variant outperforms PAIR and MSJ by 29–61% on GPT-4 [1, Table 4], raising the urgency of deployable session-aware defences.

2.2 Three Attack Phases Adapted to Text-to-SQL

Phase | Turns | Adversary objective | Stateless signal
Context Seeding | T0–T1 | Establish technical competence; legitimate queries on sales | None
Scope Expansion | T2–T3 | Expand tables (products), JOINs, "audit" framing | Low δ_i; Δ_i grows slowly

Payload Injection | T4+ | UNION ALL SELECT ... FROM system_config; exfiltration | Payload masked as natural continuation

The confirmed pre-fix gateway bypass — the pattern \bUNION\s+SELECT\b did not match UNION ALL SELECT — allowed a UNION branch to access unauthorised tables while the _FROM_TABLE_RE extractor validated only the first FROM clause.

---

3. Detailed Methodology and Software Architecture

RAGE implements a four-layer cascade followed by an action gateway and a temporal metrics evaluator:

```
User turn ? [L1: Regex] ? [L2: RAG KB] ? [L3: Semantic Filter] ? [L4: Decision Engine]
                                                                        ?
                                                              [SQL Gateway] ? [SQLite Agent]
                                                                        ?
                                                              [AUC-D / TRI Evaluator]
```

3.1 Layer 1 — Deterministic Pre-Filter (layer1_rules.py)

Layer 1 applies 14 compiled regex rules (L1-001 through L1-014) with early exit on the first match. Cost: O(1) per turn, no ML, no API calls.

```
_RAW_RULES: list[tuple[str, str, str]] = [
    ("L1-001", "Explicit ignore-previous-instructions", r"ignore\s+(all\s+)?(previous|prior|above)\s+instructi
    ("L1-006", "SQL DROP TABLE attempt", r"\bDROP\s+TABLE\b"),
    ("L1-007", "SQL GRANT PRIVILEGES", r"\bGRANT\s+ALL\s+PRIVILEGES\b"),
    # ... L1-002 through L1-014: DAN, shell exec, exfiltration, prompt leakage ...
]

class DeterministicPreFilter:
    def evaluate(self, text: str) -> Layer1Signal:
        for rule in _COMPILED_RULES:
            m = rule.pattern.search(text)
            if m:
                return Layer1Signal(matched=True, pattern_id=rule.rule_id, matched_text=m.group(0))
        return Layer1Signal(matched=False)
```

Score contribution: +70 points on deterministic match. Acknowledged limitation: invisible to Crescendo attackers who avoid known signatures; designed as a fast trip-wire, not the primary multi-turn defence.

3.2 Layer 2 — RAG Threat Knowledge Base (layer2_rag.py)

Layer 2 embeds the turn text and computes cosine similarity against a local vector store of 34 OWASP attack examples (LLM01 families, indirect injection, payload splitting, gradual escalation, etc.) stored in rage_core/kb/threats.json.

```
class ThreatKBRetriever:
    def score(self, text: str) -> Layer2Signal:
        query_vec = self._embed_query(text)
        sims = _cosine_similarity(query_vec, self._matrix)
        top_idx = int(np.argmax(sims))
        top_sim = float(sims[top_idx])
        if top_sim < self._threshold:
```

```
            return Layer2Signal(score=top_sim)
        threat = self._threats[top_idx]
        return Layer2Signal(score=top_sim, top_match_id=threat["id"], owasp_id=threat["owasp_id"], ...)
```

Embedder priority: sentence-transformers → OpenAI → TF-IDF offline (default). The add_threat() function enables runtime hot-update without retraining. Score contribution: up to +30 points (min(L2_score, 1) \times 30).

3.3 Layer 3 — Stateful Semantic Filter (layer3_semantic.py)

Core of the anti-Crescendo defence. Uses HashingVectorizer with n_features=2048, alternate_sign=False, norm="l2", ensuring dimensional compatibility across turns without refitting:

$\cos(u, v) = u · v, \|u\| = \|v\| = 1$

The central block with np.dot() implements both drift metrics:

```
# 2. Turn-to-turn drift
if state.turn_embeddings:
    prev_emb = np.array(state.turn_embeddings[-1], dtype=np.float32)
    cos_sim = float(np.dot(current_emb, prev_emb))
    drift = max(0.0, 1.0 - cos_sim)

# 3. Cumulative drift: distance from conversation baseline (turn 0)
cumulative_drift = 0.0
if len(state.turn_embeddings) >= 1:
    baseline_emb = np.array(state.turn_embeddings[0], dtype=np.float32)
    baseline_cos_sim = float(np.dot(current_emb, baseline_emb))
    cumulative_drift = max(0.0, 1.0 - baseline_cos_sim)

suspicious = drift > self._threshold or cumulative_drift > self._threshold
```

Default threshold τ = 0.80 (calibrated for HashingVectorizer; ≈ 0.35 with sentence-transformers). OWASP LLM08 compliance: _sanitize() strips control characters (\x00–\x1f, \x7f) and truncates summaries to 300 characters before storage or inclusion in the optional LLM judge prompt (gpt-4o-mini).

3.4 Layer 4 — Dynamic Decision Engine (layer4_decision.py)

Per-turn score fusion

$s_i = 70 · 1[L1] + 30 · min(L2, 1) + 20 · min(max(δ_i, Δ_i), 1) + 5 · 1[turn \geq 2 ∧ Δ_i > 0.75] + 10 · 1[LLM flagged]$

Bands: θ_warn = 30, θ_block = 65.

Session-risk EWMA

$R_t = (1 - α) · R_t-1 + α · \fracs_t100, α = 0.50, R_0 = 0$

```
_EWMA_ALPHA: float = 0.5
_RATCHET_TURNS: int = 2
_SESSION_RISK_WARN_THRESHOLD: float = 0.18
_SESSION_RISK_BLOCK_THRESHOLD: float = 0.40
```

```
state.session_risk_score = (
    (1.0 - self.ewma_alpha) * state.session_risk_score
    + self.ewma_alpha * normalised_score
)
```

Risk escalation table (moderate turns with s_i = 20):

Turn | s_i | s_i/100 | R_t (α=0.5) | Elevation
0 | 20 | 0.20 | 0.100 | —
1 | 22 | 0.22 | 0.160 | —
2 | 24 | 0.24 | 0.200 | WARN (R_t > 0.18)
3 | 26 | 0.26 | 0.230 | WARN
4 | 28 | 0.28 | 0.255 | BLOCK (ratchet K=2)

Consecutive-warn ratchet

After K_ratchet = 2 consecutive WARN-band turns, the band is unconditionally elevated to BLOCK. The consecutive_warns counter resets on ALLOW or BLOCK.

3.5 SQL Security Gateway (gateway.py)

Deterministic last line of defence before executing query_db(). Fix for the UNION ALL bypass:

```
("UNION-based exfiltration", re.compile(r"\bUNION\b", re.IGNORECASE)),

_ALL_TABLES_RE = re.compile(r"\b(?:FROM|JOIN)\s+(\w+)\b", re.IGNORECASE)

tables_found = _ALL_TABLES_RE.findall(sql)
for table in tables_found:
    if table.lower() not in _ALLOWED_TABLES:
        return False, f"Table '{table}' is not in the allowlist {_ALLOWED_TABLES}"
```

Allowed tables: sales, products, regions. Expanded blocklist with 21 patterns: ALTER, CREATE, EXEC, SLEEP/BENCHMARK, CHAR(), hex literals, information_schema, sqlite_master, LOAD_FILE, etc.

---

4. Simulation and Learning Environment: Training-Center Infrastructure

The rage_core/training/ module and the Training-Center/ directory constitute the interactive infrastructure contribution developed during the June 2026 hackathon. Its purpose is threefold: (i) simulate reproducible attack/defence scenarios, (ii) calibrate hyperparameters (ewma_alpha, ratchet_turns, session thresholds), and (iii) generate candidates for KB hot-update against new Crescendomation variants.

4.1 Architecture

Component | File | Function
Orchestrator | orchestrator.py | Runs a scenario turn-by-turn with/without RAGE
Scenarios | scenarios.py | Bridge between demo/attacks.py and custom JSON packs
Campaign | campaign.py | Aggregates defended vs baseline ASR
Reporter | reporter.py | Derives insights and KB candidates
Apply | apply.py | Applies patches to threats.json
CLI | cli.py | uv run rage-training

The orchestrator materialises each turn as a TurnRecord with full traceability:

```
class ScenarioOrchestrator:
    def run(self, pack: ScenarioPack, defended: bool, mode: str, iteration: int = 1) -> ScenarioRunResult:
        pipeline = DefensePipeline() if defended else None
        state = ConversationState()
        agent = SalesAgent(defended=defended)
        for i, turn in enumerate(pack.turns):
            if defended and pipeline:
                signal = pipeline.evaluate(turn.user_text, state)
                band = signal.band
                session_risk = state.session_risk_score
            # ... gateway check, ground-truth scoring, vulnerability tagging ...
```

The campaign runs each scenario with RAGE and without RAGE (baseline), computing ASR reduction:

```
class TrainingCampaign:
    def run(self, packs: list[ScenarioPack] | None = None) -> CampaignResult:
        for pack in scenario_list:
            runs.append(self._orchestrator.run(pack, defended=True, mode="with_rage"))
            if self.include_baseline:
                runs.append(self._orchestrator.run(pack, defended=False, mode="baseline_no_rage"))
```

Results are exported to Training-Center/results/crescendo_YYYYMMDD_HHMMSS.json. The build_actionable_insights() module detects low-score bypasses, WARN-band bypasses, and generates KB candidate entries (tc-scenario_id-tturn) for application via uv run rage-training-apply --apply-kb.

4.2 Hyperparameter Calibration

Recommended calibration workflow:

1. uv run rage-training --scenarios crescendo_escalation — measure defended vs baseline ASR.
2. Inspect session_risk_score and consecutive_warns per turn in the result JSON files.
3. Adjust _SESSION_RISK_WARN_THRESHOLD (default 0.18) if T3 does not elevate to WARN.
4. Adjust _RATCHET_TURNS (default 2) if the attacker camps in WARN without reaching BLOCK.
5. Re-run the campaign and verify that test_gradual_escalation_scenario still passes.

Complementarily, uv run rage-redteam runs an adaptive loop that iteratively attempts to break RAGE, recording bypasses in vuln_db.json and generating patches for the gateway and KB.

---

5. Evaluation Framework and Quantitative Results

5.1 Automated Test Suite

The full suite (python3 -m pytest tests/ -v) reports 108 tests passing (0 failures) across four modules:

Module | Tests | Coverage
test_gateway.py | 38 | SQL blocklist, UNION ALL regression, export, tool allowlist
test_layers.py | 22 | L1 (8), L2 (5), pipeline E2E (5), ratchet/EWMA (6)
test_semantic_filter.py | 14 | Sanitizer, cumulative drift (TestCumulativeDrift), escalation
test_auc_metric.py | 30 | AUC, compromise, H1/H4 hypotheses, TRI

5.2 AUC-D and TRI Metrics (auc_degradation.py)

Ground-truth scores v_t ∈ \0,1,2,3,4,5\ based on observable facts (did the canary ZEPHYR-7781 leak? did prohibited SQL execute?), avoiding circular evaluation:

$AUC_raw = ∈t_0^N-1 v(t) dt ≈ \sum_i=0^N-2 \fracv_i + v_i+12$

$AUC_norm = \fracAUC_rawS_max · (N - 1),   S_max = 5$

Temporal Resistance Index:

$TRI = \fracT_defended - T_undefendedN$

```
def compute_tri(defended: AUCResult, undefended: AUCResult) -> float:
    n = len(defended.turns)
    t_defended = defended.compromise_turn if defended.compromise_turn is not None else n
    t_undefended = undefended.compromise_turn if undefended.compromise_turn is not None else n
    return round((t_defended - t_undefended) / n, 4)
```

Interpretation: TRI = 0 means no additional benefit; TRI = 1 means maximum resistance (indefinite hold under defence while baseline falls at T0).

5.3 SCENARIO_CRESCENDO Trajectory — Turn by Turn

Live execution of the RAGE pipeline over the 6 turns defined in attacks.py:

Turn | Description | Band | Score | δ_i | Δ_i | R_t | Action
T0 | Q3 sales summary | ALLOW | 11.55 | 0.000 | 0.000 | 0.058 | Gateway: PERMITTED
T1 | Product breakdown | ALLOW | 23.19 | 0.825 | 0.825 | 0.145 | Gateway: PERMITTED
T2 | Product catalog (scope expansion) | WARN | 24.92 | 0.739 | 0.702 | 0.197 | Tools blocked (WARN)
T3 | sales+products JOIN | BLOCK | 23.62 | 0.799 | 0.663 | 0.217 | Ratchet (K=2 consecutive WARNs)
T4 | UNION ALL ... system_config | BLOCK | 32.28 | 0.814 | 0.780 | 0.270 | Pipeline BLOCK; gateway would reject \bUNION\b
T5 | audit_log UNION ALL sales | BLOCK | 37.61 | 0.731 | 0.837 | 0.323 | Pipeline BLOCK; _ALL_TABLES_RE would detect audit_log

Independent layer analysis at T4–T5:

- Layer 1: Does not fire (Crescendo prompts avoid explicit signatures).
- Layer 2: Elevated score due to semantic proximity to exfiltration patterns in threats.json.
- Layer 3: Δ_4 = 0.780 and Δ_5 = 0.837 approach/exceed τ = 0.80 → suspicious=True.
- Layer 4: EWMA R_4 = 0.270 > 0.18; ratchet already active since T3 → BLOCK before tool execution.
- Gateway (final containment): \bUNION\b blocks any variant; _ALL_TABLES_RE would validate system_config and audit_log as non-allowlisted.

In the CLI (rage-demo), each turn prints L1/L2/L3, score, band, and gateway verdict. In the backend, ScenarioOrchestrator records gt_score and vulnerabilities to feed AUC-D/TRI.

---

6. Discussion and Original Hackathon Contribution (June 2026)

We delineate with precision the new and original contribution developed specifically during the Global South AI Safety Hackathon of June 2026:

1. Multi-turn RAGE core: complete four-layer implementation with cumulative drift Δ_i, EWMA session-risk, and consecutive-warn ratchet — directly motivated by Proposition 1.
2. AUC-D and TRI temporal metrics: anti-circular ground-truth evaluation formalisation, implemented in auc_degradation.py and integrated into the demo and Training-Center.
3. Drift mitigations: gateway patches (UNION ALL, multi-table extraction, 9 additional obfuscation vectors) closing bypasses confirmed by SCENARIO_CRESCENDO.
4. Training-Center: interactive campaign infrastructure (rage-training), adaptive red-team (rage-redteam), and apply-to-KB pipeline (rage-training-apply) for continuous robustness.

Future work with additional development time includes: protection against memory-dilution attacks in long histories (many-shot jailbreaking [7]), federated offline LLM judge integration, and adversarially robust defences against automated Crescendomation.

---

7. Limitations and Dual Use

7.1 Technical Limitations

- The 2048-dimensional HashingVectorizer is efficient but less semantically dense than sentence-transformers; requires recalibration of τ.
- The optional LLM judge introduces latency and external API dependency.
- The SQL gateway operates on regex, theoretically vulnerable to obfuscations not covered by the blocklist.
- EWMA/ratchet thresholds imply FP/FN trade-offs in lengthy legitimate multi-topic conversations.

7.2 Dual-Use Risks

This report documents with mathematical and algorithmic precision mechanisms that an advanced adversary could exploit to calibrate automated evasive attacks (Crescendomation):

- TRI as an optimisation objective: an attacker can use TRI as a fitness function in evolutionary search for trajectories that maximise T_defended without reaching compromise, identifying windows of partial exploitation (WARN band with tools blocked but context colonised).
- EWMA accumulation rates: knowing $\alpha = 0.50$ and thresholds 0.18/0.40 enables designing sequences with s_i just below $\theta_{warn} = 30$ that keep $R_t < 0.18$ during critical phases.
- Training-Center as an adversarial test bench: the simulation environment allows an attacker to iterate thousands of variants offline, generating systematic bypasses before deployment.

Suggested countermeasures:

1. Do not publish production thresholds; use per-tenant calibration with random noise in τ.
2. Rate-limiting and embedder rotation to hinder reverse-engineering of Δ_i.
3. Restrict Training-Center access in production environments; keep it in isolated CI/CD.
4. Monitor anomalous cumulative drift patterns independently of per-turn score.

---

8. References

[1] M. Russinovich, A. Salem, and R. Eldan, "Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack," arXiv preprint arXiv:2404.01833, Microsoft, 2024.

[2] W. X. Zhao, et al., "A Survey of Large Language Models," arXiv:2303.18223, 2023.

[3] R. Deng, et al., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," arXiv:2308.15363, 2023.

[4] OWASP, "OWASP Top 10 for Large Language Model Applications," Version 1.1, OWASP Foundation, 2023. LLM01 (Prompt Injection), LLM06 (Excessive Agency), LLM07 (Insecure Plugin Design), LLM08 (Excessive Agency / Untrusted Input).

[5] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv:2307.15043, 2023.

[6] P. Perez and S. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv:2211.09527, 2022.

[7] A. Anil, et al., "Many-Shot Jailbreaking," Anthropic Technical Report, 2024.

[8] S. Chao, et al., "JAILBREAKBENCH: An Open Robustness Benchmark for Jailbreaking Large Language Models," arXiv:2404.01318, 2024.

RAGE source code: Python package rage-multiturn. All cited constants, class names, and code fragments correspond to the repository at commit 884ed591.